Chapter 6

Object-Oriented Programming

Object-oriented programming (OOP) is the methodology and technology of representing data or concepts as real-world objects in computer programming. A software object has properties that describe the object as well as behaviors that perform tasks. For example, a car object may be described by properties such as maker, model, color, engine horsepower, etc. A car object may have behaviors such as to start, to move, and to stop. By modeling many related objects, such as cars, drivers, roads, and signals, the operations and behaviors a complex system, such as a transportation system, can be simulated by OOP.

In object-oriented programming, objects are created from particularly designed templates called classes. Technically, a class is a data structure that defines the data members and methods (i.e. procedures) that process data. Before introducing object-oriented programming, this chapter will first introduce a user-defined data structure named "Structure" in VB programming. Then it will discuss three key concepts and techniques of object-oriented programming: encapsulation, inheritance, and polymorphism. Examples will be used to demonstrate the process of class definition, object instantiation, and object manipulation in computation. In the end, two other important concepts, namespace and enumeration, are introduced.

1. Structure

Boolean, Integer, Single, and Double are simple data types built-in the programming language of VB. A structure is a user-defined complex data type that is made up of multiple data members. It can be defined either inside or outside a class, but it must not be defined inside a procedure. The syntax for a structure is:

```
Structure StructureName
     Dim Member1 As <Data type>
     Dim Member2 As <Data type>
     (other members...)
End Structure
```

For example, you can define a structure named *Address* that consists of four members including street, city, state, and zip code.

```
Structure Address
    Dim Street As String
    Dim City As String
    Dim State As String
    Dim Zip As String
End Structure
```

Once an Address structure is defined in your program, you can use it to declare address variables same as using a simple data type. For example,

```
Dim addr1 As Address    ' declares an address variable from the structure
Dim addr2 As Address    ' declares another address variable
```

To access a member of a structure variable, you need to use the variable name followed the member name with a dot (.) between them to separate the two parts. For example, to assign values to members of variable *addr1*,

```
addr1.State = "123 S. Beaver St."
addr1.City = "Flagstaff"
addr1.State = "AZ"
addr1.Zip = "86001"
```

Like simple data type variables, structures can be used to declare array variables. For example, you can use the Address structure to create an array of addresses.

```
Dim addr(9) As Address  ' declares an array for 10 addresses
```

The following code assigns values to addr(0)

```
addr(0).State = "123 S. Beaver St."
addr(0).City = "Flagstaff"
addr(0).State = "AZ"
addr(0).Zip = "86001"
```

A member of a structure may be defined by another structure as well. For example, you can define structure named Student in which a home address member is defined by the Address structure.

```
Structure Student
    Dim FirstName As String
    Dim LastName As String
    Dim HomeAddress As Address       ' note: this member is a structure
    Dim DateOfBirth As Date
    Dim English As Integer
    Dim Math As Integer
    Dim Science As Integer
End Structure
```

If you declare a student variable named *pupil* use code

```
Dim pupil As Student    ' declares a student variable
```

Then the address of the pupil can be accessed as the following.

```
pupil.HomeAddress.Street
pupil.HomeAddress.City
pupil.HomeAddress.State
pupil.HomeAddress.Zip
```

2. Object-Oriented programming

In traditional programming, every piece of data is hold by an individual variable, so that large projects often need to use many variables to keep track of various information. Maintaining a large number of variables is not only a burden to programmers but also subjects to errors. By encapsulating certain related information inside one unit, structures can enhance programming efficiency by reducing the number of loose individual variables. Moreover, structures allow some data to mimic natural characteristics of real-world things. For example, a real-world address is composed of several parts, and a student does have various properties. It is much more natural and convenient to treat one address or one student as one thing than as a bunch of loose variables.

Object-oriented programming is the methodology and technology of representing data or concepts as real-world objects using programs. Conceptually, an object is like a natural object: it has *properties* that describe the object and it has behaviors that carry out tasks. Technically, a object is similar to but more advanced than a structure since it packs not only data members but also procedures that process data in a block of code. Properties are data members, and methods are procedures (methods) that process data.

In the 1980's, a paradigm shift took place in computer science, which has virtually set object-oriented programming as a standard for software engineering. Today, most programming languages support object-oriented programming.

In object-oriented programming, an object is often created from a class. For example, you created open file dialog object from the *OpenFiledialog* class in Chapter t. Therefore, a class can sometimes be regarded as a template that makes objects, and an object is commonly referred to as an instance of a class. In computer science terms, to create an object from a class is often called to instantiate the class. In fact, all the windows forms and controls are created from classes designed and programmed by Microsoft engineers. When you pick up a button from the VB toolbox and drag a button control on a form, the VB IDE creates a few lines of code that uses a button class to create the button object for you. Moreover, all the ArcObjects components are classes designed and programmed by the ESRI engineers for making the ArcGIS software products.

Object-Oriented programming derives its power for modeling complex systems from the following three key concepts and techniques:

- Encapsulation
- Inheritance
- Polymorphism

2.1 Encapsulation

Encapsulation is a technique that packs and hides all data members and data processing procedures of an object inside a shell while allowing some data members and procedures to be exposed. By definition, an exposed data member is called a *property*, and an exposed procedure (sub or function) is called a *method*. The assembly of properties and methods of an object is referred to as the *interface* of the object. Application developers can manipulate an object through its properties and methods.

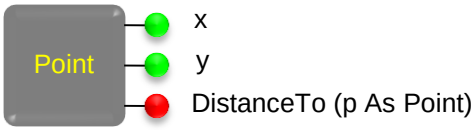


Figure 1 Class encapsulation

Figure 1 pictorially describes a point object that may be used in GIS applications. This object hides the mechanism that maintains its data members and procedures that process data in a shell. But it exposes an *x* property, a *y* property, and a *DistanceTo*() method for application developers to interact when creating applications. An application developer does not have to know how the object works internally. Knowing the properties and methods of an object is sufficient for a developer to use the object.

An analogy of encapsulation is a wrist watch. The watch uses a steel case to hold a very complex machine consisting gears and springs in order to protect the delicate machine from being spoiled. A watch exposes a dial with three hands and one handle. The hands which tells time in hours, minutes, and seconds are similar to properties, and the handle of the watch can be compared to a method since it can be used to perform tasks such as to tighten the spring and adjust time. A user does not have to know what are inside and how the machine works. The user uses the watch to read time and use the handle to keep it running properly.

In programming, if we know the interface of an object, we can use it in application development. For example, if there is an object named *map* which has properties such as background color, extent, and map scale, as well as methods such as adding layer, zooming in, and zooming out, then we can use this object in applications that need to display maps.

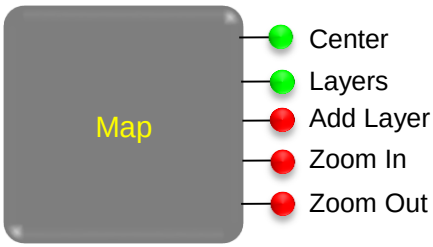


Figure 2 An imagined Map class

If available, such a map object can be very useful to us. In fact, Figure 2 is part of the of the map class interface of the leaflet open-source JavaScript library for Internet mapping (http://leafletjs.com/reference.html). And the ESRI ArcObjects library provides a more sophisticated map component with a large number of properties and methods.

2.2 Inheritance

Inheritance is a technique that allows one class to be defined based on another. This establishes a parent-child relationship between classes in which a child class inherits all properties and methods from its parent class. A child class can have additional

properties and methods to define its own unique nature. Inheritance allows classes to be organized in a hierarchical structure. For example, if a Circle class can be defined based on the Point class, which establishes an inheritance relationship between the two classes (Figure 3). Inheritance enables OOP the power for modeling complex systems in the real-world.

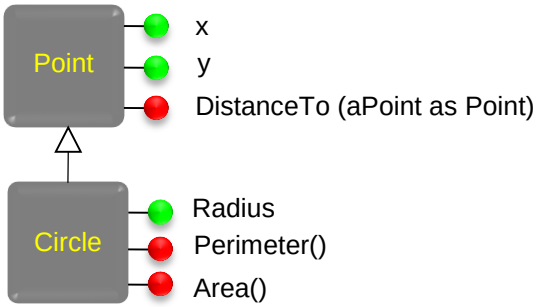


Figure 3 Class inheritance

An inheritance relationship is a parent-child relationship in which the child class inherits all properties and methods of the parent class. In the case if Figure 3, the Circle class is a child class of the Point class, so it inherits all properties and methods of its parent. This means that the Circle class also has the *x* and *y* properties and *DistanceTo*() method. In GIS, you can find many examples of inheritance. For example, a Geometry class is inherited by a Point, a Line, a Polygon, etc., and a layer class is inherited by a feature layer class and a raster layer class.

2.3 Polymorphism

Polymorphism is mechanism that allows a class to have multiple methods for the same purpose to use a same name, which is called *method overloading*, or a child class to have methods that are already provided by its parent class, which is called *method overriding*. In method overloading, each method (procedure) must have a different set of parameters that differ either in number or in data type of parameters (Figure 4).

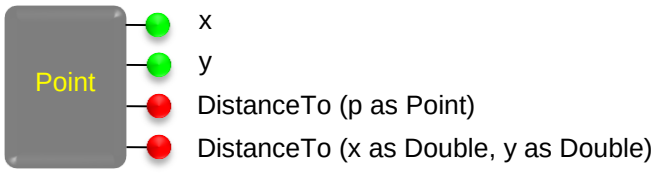


Figure 4 Class polymorphism (overloading)

Polymorphism adds intelligence to objects by allowing an object to take proper actions depending on the user input. In Figure 4, the Point class has two overloading *DistanceTo*() methods both intended for measuring distances. But each has a different parameter set: the first DistanceTo() method declares one parameter declared as Point type, and the second method declares two parameters both as Double type. With two DistanceTo() methods, a point object can compute distance with different user inputs. If a client procedure calls the DistanceTo() method of a current point object by providing another point object as argument, the first function will be automatically invoked to

calculate distance between the current point object and the argument point object. Otherwise, if a client procedure calls the DistanceTo() method of a current point object by providing a pair of x and y coordinates as arguments, the second function will be automatically executed to calculate distance between the current point object and a location described by the x and y coordinates.

In windows programming, some built-in objects have more than a dozen overloading methods. Without polymorphism, each overloaded method must have a unique name in a class although all do the same job. With polymorphism, one method of an object can carry out a task based on a variety of arguments provided to it. Technically, the object determines an appropriate overloaded method based on the number and data type of incoming arguments.

3. Class Structure

In object-oriented programming, elements of a class, including data members and procedures, are encapsulated in a class block. The following sample code shows the structure and some elements of a typical class.

```
Public Class ClassName
    ' declare necessary private data members (private class-level variables)
    ' these variables are only available inside the class
    Private variable1 As DataType
    Private variable2 As DataType
    ...

    ' declare necessary public data members, i.e. properties
    ' these variables can be accessed inside and outside the class
    Public variable3 As DataType
    Public variable4 As DataType
    ...

    ' create a class constructor (optional)
    ' the constructor is invoked when a new object is instantiated
    Public Sub New()
        ...
    End Sub

    ' a public procedure is a method
    ' it can be called by any procedure inside this class and
    ' can be accessed by clients outside this class
    Public Function Func2 (...) as DataType
        ...
    End Function

    ' a private procedure is not exposed to the outside of the class
    ' it can be called by any other procedures inside this class
    Private Function Func1 (...) as DataType
        ...
    End Function
End Class
```

In a class block, keywords *Private* and *Public* are essential in determining variable scope. Class-level variables, both private and public, are available to all procedures inside the class block. However, class-level variables declared with the Private keyword can only be used inside the class block and cannot be seen or accessed by code outside the current class (i.e. by another class). Class-level variables declared with the Public keyword are not only available to procedures inside the current class block, they are also available to external clients (other classes). In fact, class-level variables declared with the Public keyword are *properties* of the class.

By the same token, procedures declared with the Private keyword can only be used by code inside the current class and are invisible to external clients, but procedures that are declared with the Public keywords can not only be used by code inside the current class block but also available to external classes. By definition, procedures declared with the Public keyword are *methods* of the class.

Sub procedure New() is a special procedure called the class *constructor*. When a client uses the keyword New to instantiate an object from this class, this New() procedure is executed. So it is somewhat similar to the form Load event procedure. The constructor can be used to initialize certain variables or perform some initial tasks. In VB programming, this New() procedure is optional.

4. Example: PointCircle Project

This section will use examples to demonstrate object-oriented programming by creating a simple Point class and a Circle class that inherits the Point class. Then point objects and circle objects will be instantiated from these classes in these examples. The three fundamental object-oriented programming techniques including encapsulation, inheritance, and polymorphism will be demonstrated.

4.1 Creating Classes and Objects

A point is the simplest geometric primitive in GIS. In programming, a Point class can be simply defined as:

```
Public Class Point
   Public x as double
   Public y as double
End Class
```

Please follow steps below to create a Point class in a VB project.

1) Create a new Windows application in VB and name the project “PointCircle”.
2) In the Solution Explorer, right click on the project name, then select Add >> Class... to open the Add New Item dialog.
3) Enter a file name “Point.vb” for the class module (file) in the name box at the bottom and click “Add” to create a Point new class.

As you follow this book so far, you have been writing code in the main form class module. Now you have created an additional class module Point.vb in a project. In this module, an empty Point class block has been created. You can type in the following two lines of code inside the Point class block:

```
Public x as double
Public y as double
```

Now, a Point class is ready for use to instantiate point objects although it is so simple as to only have two properties without methods. But it is enough for us to create objects.

4) Create a button named "btnPointTest" on the main form (e.g. Form1) and set the Text property of the button with value "Point Test".
5) In the Click event procedure of the point test button in the Form1 class module, write the following code:

```
Dim p1 As Point   ' 1. Declares a variable of the Point type
p1 = New Point()
     2. Creates a new point object
        3. Stores the address of the point object in variable p1
```

The first line declares a point type variable *p1* which will be used to hold the address of a point object. This line does not really create an object. The second line does two things: first, it instantiates a new point object by using code `New Point()` (to the right of the equal sign), and then it assigns the address of the new point object to variable *p1*. Since variable *p1* holds the address of the new point object, it is a representative of the object. Whenever variable *p1* is called later, it leads to that point object. Conceptually, you may regard *p1* as the point object itself although the variable only carries the address of the object. In practice, the two lines of code above can often be combined into one line as the following.

```
Dim p1 As Point = New Point()
```

or simply as,

```
Dim p1 As New Point()
```

In programming terms, a point object is an instance of the Point class. A class cannot be used as data to be processed in computations, only instances of a class, i.e. objects created from classes, are actual data that can be manipulated in computation. A class is a template such as a cookie cutter or a blueprint of a house. But an object is a concrete thing, such as a cookie or a house, created from the template or blueprint. We cannot eat a cookie cutter but cookies made by a cookie cutter, nor can we live in a blueprint but a house built from it.

6) After creating object *p1* from the Point class, you can set coordinates to its *x* and *y* properties. For example,

```
p1.x = 1
p1.y = 1
```

Then you can create another point object (p2) and set its x and y properties.

```
Dim p2 As New Point()
p2.x = 5
p2.y = 4
```

7) You will create a method named DistanceTo() in the Point class, which will allow a point object to measure distances between itself and another point. To do so, switch to the Point.vb class module, and add the follow public function into the Point class. You can put the function after the declarations of x and y properties.

```
Public Function DistanceTo(pIn As Point) As Double
    Dim px As Double = pIn.x    ' gets x coordinate of the incoming point
    Dim py As Double = pIn.y    ' gets y coordinate of the incoming point
    Dim dist As Double = Math.Sqrt((x - px) ^ 2 + (y - py) ^ 2)
    Return dist    ' returns the result
End Function
```

This method declares one parameter (*pIn*) of the Point type. Therefore, when a client calls the method to calculate distance, it must pass a point object to the method as an argument (input). Inside this function, the first two lines get the x and y coordinates of the incoming point received by parameter *pIn* and store the coordinates in variables *px* and *py*. Then the third line calculates distance between the coordinates (x and y) of the current point object itself and the coordinates (*px* and *py*) of the incoming point object. Finally, the function returns a Double type distance value.

Since function DistanceTo() is declared as a Public function, it is available to code inside and outside the Point class. Next step will show you that the DistanceTo() method can be accessed in the button click event procedure in the main form (Form1) class.

8) Switch to the main form class (Form1) module and add the following code to the end of the point test button's Click event procedure.

```
Dim d As Double = p1.DistanceTo(p2)
MessageBox.Show("Distance between p1 and p2 is: " & CStr(d))
```

The first line calls the DistanceTo() method of object *p1* and passes object *p2* as an argument to it since the method requires a point object as input. The result returned from the DistanceTo() method is then assigned to a new variable *d* which is displayed in a message box in the second line. In fact, the same job can also be done by calling the DistanceTo() method of point *p2* and passing *p1* to it as an argument, i.e.

```
d = p2.DistanceTo(p1)
```

You may have noticed as you type in the above code, as you type the dot (.) symbol after variable *p1* or *p2*, the x / y properties and the DistanceTo() method you created in the Point class are displayed in the IntelliSense. This indicates that the public properties and method of the Point class are exposed to the client module, i.e. the Form1 class. If you change any of the domain keywords of x, y or DistanceTo() from Public to Private in

the Point class, you will not be able to see or use that property or method in the Form1 class. Try it!

Conceptually, classes and objects may be considered to exist on separate layers: classes reside on a lower layer and objects which are instances of classes exist on an upper layer (Figure 5). In the application layer, when an object is instantiated from a class, the constructor (Sub New()) of the class on the lass layer is executed. When the application layer calls a method of an object to carry out a task, the method on the class layer is executed.

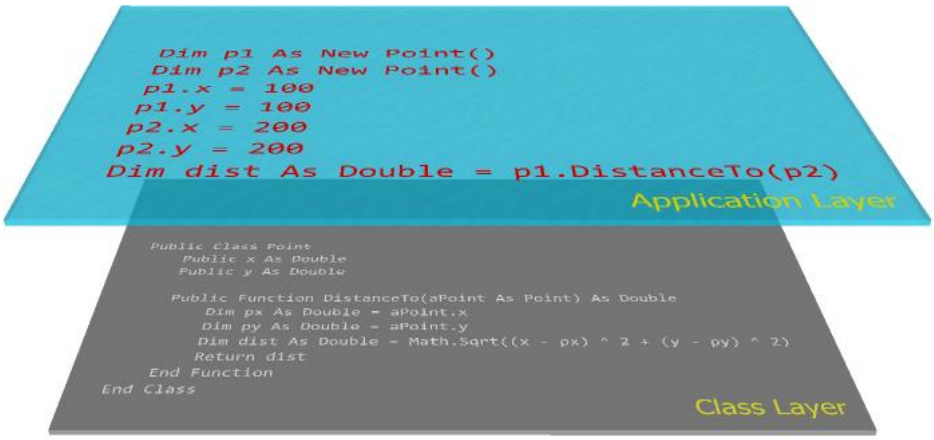


Figure 5 Class and object on different layers

To understand how classes and objects work and how the PointCircle program flows, you can set a break point at the line “Dim p1 As …”. Then run the program and use the F8 or F11 key to step through the whole process. You will find how point objects are created from the Point class, how values are assigned to properties, how the DistanceTo() method is called and executed to compute distance, as well as how a distance value is returned to the client.

Object-oriented programming make it possible for some developers to specialize in creating tools or applications using components (classes) created by others. While component developers make foundational components (classes) which may be distributed as software development kits, application developers can focus on using the foundational components to create tools and applications. This is very similar to the construction industry. While home component manufacturing companies specialize in creating home components such as windows, doors, and walls, home builders can focus on designing homes and building houses by using available components. In GIS, ESRI engineers have created a full package of GIS components for the ArcGIS software family – ArcObjects. These components allow application developers including us to program specialized tools and even comprehensive GIS software products!

4.2 Property procedure

Two-way properties (properties that allow read and write) can simply be declared as public data members in a class. But properties declared that way does not support validation when the user sets values. Crash could happen if an invalid value is set to a property. VB.NET provides a refined property procedure which declares properties that can perform some data processing such as validation of user input. More importantly,

property procedure can also define one-way (read-only or write-only) properties. The syntax of the property procedure is as the following.

```
Private var As <data type> ' declares a private data member in the class

Public Property PropertyName() As <data type>
    Set(value As <data type>)
        var = value        ' assigns a user-provided value to var
    End Set
    Get
        Return var         ' returns a value
    End Get
End Property
```

With property procedure, a public property with a given name provides a channel to a private data member (i.e. a private class-level variable) in the class. For the Point class you created in the PointCircle project, you may use property procedures to define the x and y properties. For example, an x property procedure can be coded as the following.

```
Private _x As Double  ' declares a private data member _x in the class

Public Property X() As Double
    Set(value As Double)
        _x = value    ' assigns a user-provided value to member _x
    End Set
    Get
        Return _x     ' returns the value of _x
    End Get
End Property
```

As a convention in programming, an underscore symbol is often used to prefix a private data member name. In this example, the private data member is named "_x" and the property is named "X". The Get block in the property procedure simply returns the value of the private data member _x to the client. The Set block assigns a user provided value (received by parameter *value*) to the private data member _x. Data validation can be implemented in this block if necessary. For example, suppose an application requires that the x coordinate must not be negative, to validate the user input while setting the X property, the following code can be used inside the Set block.

```
If value >= 0 Then
    _x = value     ' assigns a user-provided value to _x
Else
    ' handle error ...
End If
```

To define a read-only property, you can declare the property procedure with keyword ReadOnly and use the Get block only inside the property procedure. To define a write-only property, you can declare the property procedure with keyword WriteOnly and use the Set block only inside the property procedure. For example, you can create a write-only property named "SpatialRef" in the Point class to allow the user to set a spatial reference to point objects.

```
Private _sr As String      ' declares a private data member

Public WriteOnly Property SpatialRef () As String
    Set(value As String)
        _sr = value
    End Set
End Property
```

4.3 Inheritance

In graphics programming, a Circle class is often created based on a Point class. By inheriting the Point class, a Circle class can take on the x and y coordinate properties so that we do not need to declare new properties for the circle center. A Circle class can also take on the DistanceTo() method so that we can easily measure distances between circle objects. In addition, a Circle class can have its own properties such as Radius as well as its own methods such as computing perimeter and area.

To create a Circle class, you can create a new class module (a new vb file) by following the steps that you use create the Point class to, or you may simply write code in an existing module in the current project. In this example, I would like you to try the second method to create a new Circle class in the existing Point class module. Please enter the following code into the Point.vb module just below the Point class block.

```
Public Class Circle
    Inherits Point  ' defines that the Circle class inherits the Point class

    ' public data member
    Public Radius As Double

    ' create a Perimeter method
    Public Function Perimeter() As Double
        Return Math.PI * Radius * 2
    End Function

    ' create an Area method
    Public Function Area() As Double
        Return Math.PI * Radius * Radius
    End Function
End Class
```

Inside the Circle class, the first statement “Inherits Point” makes the class to inherit the Point class. To test the Circle class, please switch to the main form class module and create a new button named “btnCircleTest” with caption “Circle Test” on the form. Then implement the click event procedure of the button by using the following code.

```
Private Sub btnCircleTest_Click(...) Handles btnCircleTest.Click
    Dim circle1 As New Circle()     ' instantiates a circle object
    Dim circle2 As New Circle()     ' instantiates another circle

    ' sets properties of circle1
    circle1.Radius = 5
    circle1.x = 0
```

```
        circle1.y = 0

        MessageBox.Show("Perimeter of circle1 = " & CStr(circle1.Perimeter()) & _
                        Chr(13) & "Area of circle1 = " & CStr(circle1.Area()))

        ' sets propertyes of circle2
        circle2.Radius = 10
        circle2.x = 100
        circle2.y = 100

        MessageBox.Show("Perimeter of circle2 = " & CStr(circle2.Perimeter()) & _
                        Chr(13) & "Area of circle2 = " & CStr(circle2.Area()))

        ' calculate distance between circles
        Dim dist As Double = circle1.DistanceTo(circle2)
        MessageBox.Show("Distance between the two circles = " & CStr(dist))
    End Sub
```

You did not create the x and y properties for the Circle class, but you are able to use these properties of the circle instances in the client procedure (i.e. the button's Click event procedure in the main form class). This is because the Circle class has inherited the x and y properties from the Point class.

If you wonder how the code works, you can analyze the program flow by setting a break point at the header of the button's Click event procedure and then use F8 to step through the code.

4.4 Polymorphism

1) Overloading the DistanceTo() method

So far when you call the DistanceTo() method of the Point class you must pass a point object to it. It would be more convenient if the DistanceTo() method can also accept x, y coordinates as arguments in case a point object is not available or not necessary. This is made possible by polymorphism which allows us to overload the DistanceTo() method in the Point class. In other words, polymorphism allows us to create another DistanceTo() method in the Point class that takes two arguments (x and y). To do so, please add the following function into the Point class.

```
    Public Function DistanceTo(x As Double, y As Double) As Double
        Dim dist As Double = Math.Sqrt((Me.x - x) ^ 2 + (Me.y - y) ^ 2)
        Return dist
    End Function
```

This method declares two parameters x and y to accept two coordinate values passed in from the client. However, the current point object itself has two member variables also named x and y. Inside the DistanceTo() function, therefore, two different sets of x and y variables are available, which can cause confusion to the computer. One solution to this variable naming conflict could be to rename the function parameters, for example, to use names *argx*, *argy* for the parameters. However, users are used to the simple names x and y for coordinates. To distinguish these two sets of x and y variables, VB allows the member x and y of the class itself to be referenced by Me.x and Me.y where keyword

"Me" refers to the current Point class itself, so that we can still use the simple names x and y for the parameters.

2) Overloading of the constructor

Every class has a special method named New(). This method is called the constructor of the class. The constructor is executed automatically whenever a new object is instantiated from the class with keyword New. By default, the constructor does not have parameter and it contains no action code. Since the constructor is automatically executed in object instantiation, it can be used to carry out certain initialization or preprocessing tasks. The constructor is often not present in the class block when it contains no code. For example, the constructor is not present in the Point class.

To instantiate a point object from the Point class using the following code, no argument needs to be provided because the default constructor does not have parameters.

```
Dim p As New Point()
```

Once a new point object is instantiated, two additional lines of code must be used to set the x and y properties. It would be more convenient if the Point class can set the x and y properties simultaneously while instantiating objects. This can be implemented by overloading the constructor. To do so, you can create another New() sub procedure that declares two parameters for coordinates in the Point class. It is important to note that to overload the constructor, the default constructor must be present in the class block, even if it contains no code. The following shows how the constructor of the Point class is overloaded.

```
Public Class Point
    Public x As Double      ' data member x
    Public y As Double      ' data member y

    Public Sub New()        ' the default constructor
        ' leave it blank
    End Sub

    Public Sub New(x as Double, Y as Double) ' an overloading constructor
        Me.x = x    ' assigns argument x to data member x
        Me.y = y    ' assigns argument y to data member y
    End Sub
End Class
```

With two constructors are present in the Point class, the user can choose either one to instantiate point objects. In the client class (form class), if the user instantiates a new point object with a pair of values, for example,

```
Dim p As New Point(5, 4)
```

the second constructor will be automatically invoked, and the argument values are then passed to Me.x and Me.y which are the data members x and y. This saves two lines of code in the client procedure and makes the code looking neat as well.

5. Namespace

You have seen and used namespaces such as IO, IO.File, and Math in previous chapters. And you know that namespaces are unique names used to categorize resources such as built-in classes and functions. In VB programming you can define your own namespaces and use them to organize your project resources including classes, procedures, and variables so as to avoid naming conflict with other resources. For example, Microsoft has a Point class which is placed under namespace System.Drawing. To avoid possible confusion or conflicts between your Point class and the Microsoft Point class, you may define a unique namespace for your Point class.

You can create a namespace called “MyGIS” for the Point and Circle classes. To do so you just need to place the two classes inside a namespace block.

```
Namespace MyGIS
    'Place your Point and Circle classes inside this block
End Namespace
```

After defining a namespace for the Point and Circle classes, you need to make changes in the client code (the form class) by referring the Point class to as MyGIS.Point and the Circle class as MyGIS.Circle. For example,

```
Dim p1 As MyGIS.Point = New MyGIS.Point()
Dim circle1 As New MyGIS.Circle()
```

Moreover, namespaces can be nested to form a hierarchical structure. For example,

```
Namespace MyGIS
    ' some resources ...

    Namespace MyGeometry
        ' some resources...

        Namespace MyTopology
            ' some resources...
        End Namespace
    End Namespace

    Namespace MyTable
        ' some resources...
    End Namespace
End Namespace
```

A hierarchical system gives you great convenience in organizing your classes in large software development projects. You can place your classes in any namespace at any level. A class existing in a hierarchical namespace system can be referenced by the full path of namespace which is a series of namespace names separated by dot (.) signs. For example, if the Point class is placed inside MyGeometry under the MyGIS namespace, it can be referenced by MyGIS.MyGeometry.Point.

In addition to using the namespace structure, namespaces are also defined by the module structure in VB.NET. A module is a standalone VB file in a project. It is often used to hold project-wide variables and procedures. For example, a module with file named “ProjectStuff.vb” may contain the following code:

```
Module ProjectStuff
    Public ZoomLevel As Double
    ' other project-wide variables...
    ' some useful procedures...
End Module
```

In this example, module *ProjectStuff* is also a namespace. Variable *ZoomLevel* can be accessed from any module in the same project through *ProjectStuff.ZoomLevel*.

Furthermore, each VB.NET project is automatically assigned a root namespace with the name of the project. Everything including classes, modules, and namespaces created in the project are under the root namespace.

6. Enumeration

An enumeration is data structure that defines a list of constant values. As a data type, enumerations can be used to define variables. If a variable is declared from an enumeration, IntelliSense can assist the programmer to select a proper values from a list in value assignment.

In Section 4.2, you created a write-only property SpatialRef to allow the user to specify a spatial reference to point objects. Since the property was declared as String data type, it is possible that the user may assign arbitrary and inconsistent values to it. For example,

```
p1.SpatialRef = "GCS"
p2.SpatialRef = "Geographic Coordinate System"
```

To avoid arbitrary or inconsistent values for a variable, an enumeration can be used to define a list of constant values. An enumeration is declared with the Enum block using the following syntax:

```
Enum <enumerationname> [As <an integer data type>]
   Member1 = <a unique integer number>
   Member2 = <a unique integer number>
   (more members ...)
End Enum
```

To create an enumeration of acceptable spatial references for your project, you can add the following code in the MyGIS namespace in the Point.vb code module like a class.

```
Public Enum SpatialReference As Byte
    GCS = 0          ' Geographic Coordinate System
    UTM = 1          ' Universal Transverse Mercator coordinate system
    SPCS = 2         ' State Plane Coordinate System
    MGRS = 3         ' Military Grid Reference System
```

```
        Custom = 4
    End Enum
```

The block of code above defines an enumeration that contains a number of spatial reference names to be used in the project. An enumeration can be placed in any code module like a class. You may create a separate module to hold your enumerations or you may place an enumeration in an existing class module. Like classes, enumerations can also be placed in namespaces. Sometimes, enumerations may need to be declared inside classes.

Once you have created a spatial reference enumeration in the PointCircle project, you can use it to define variables. For example, you can change the declaration of member variable _sr in the Point class to

```
    Private _sr As SpatialReference
```

And then change the write-only property declaration as the following

```
    Public WriteOnly Property SpatialRef () As SpatialReference
        Set(value As SpatialReference)
            _sr = value
        End Set
    End Property
```

After making these changes in the Point class, you can assign spatial references to the point objects created in the button's Click event procedure in the form class. For example,

```
    p1.SpatialRef = MyGIS.SpatialReference.Custom
```

When you enter the above code, you should notice that you can select a spatial reference from a list in IntelliSense after you type the equal sign, just like when you are setting a color to a control.

Summary and Highlights

A structure is user-defined complex data type composed of a number of data members. A class is a more sophisticated data type representing a real-world thing that has properties and behaviors. A class encapsulates data members and procedures that process data. In a class, public data members are properties and public procedures are methods. Properties and methods of a class are exposed for access by external classes. A class is sometimes regarded as a template that can creates objects, and conversely, an object is an instance of a class. In a program objects are actual data and computation is performed on objects instead of classes.

Object-oriented programming is a methodology as well as a technology that represents data as real-world objects using programming. Object-oriented programming derives its

power for modeling complex systems from three techniques: encapsulation, inheritance, and polymorphism. Encapsulation allows classes to hide all data members and procedures in a shell while allowing some of them to be exposed. Inheritance allows one class to inherit another, so that the child class inherits all properties and methods of the parent class. Inheritance enables OOP to model complex systems in the real world. Polymorphism allows a class to overload and override methods. In method overloading, multiple methods use the same name, but each method must have different number or types of parameters. In method overriding, a child class can have methods that override methods of its parent class. Polymorphism enables an object to adapt to environment.

Namespace is a mechanism that uses unique names to organize resources in a hierarchical system. A namespace can be defined by a namespace block. A module in a VB project also defines a namespace. All resources in a project are under a root namespace of the project.

An enumeration is a data type that defines a limited list of constant values. It can be used to define a variable with a list of permissible values so as to avoid arbitrary or inconsistent values assignment.

Exercises

1. Answer the following questions.
   1) What is a structure?
   2) How do you create a structure?
   3) How do you declare variables from structures?
   4) How do you assign values to structure members?
   5) What is a class?
   6) What is an object?
   7) What is an instance of a class?
   8) How do you create objects from a class?
   9) What is the relationship between classes and objects?
   10) What is a property?
   11) What is a method?
   12) What does encapsulation mean?
   13) What does inheritance mean?
   14) How does one class inherit another class?
   15) What does polymorphism men?
   16) What is overloading?
   17) What is overriding?
   18) What is a constructor of a class?
   19) How do you define a read-only property?
   20) How do you define a write-only property?
   21) What is a namespace?
   22) How do you define namespaces for classes you create?

For problem 2 through 6, please write down your code in a text file or Microsoft World document.

2. Create a class named *AddingMachine* that has two Double type properties *Number1* and *Number2* and two methods named *Sum* and *Difference* that calculate the sum and difference of the two data members.

3. Create a class named *MPG* that will be used to calculate gas mileage. The class has two Single type properties *Miles* and *Gallons*, and a method named *Calculate* that returns a Single type mile per gallon result.

4. Create a class named *Sphere* that calculates the volume and surface area. The class must have a property *Radius* of Double data type, a method *Volume* that returns the volume and a method *Surface* that returns the surface area.
   (Note: volume of a sphere is calculated by $v = \frac{3}{4} \pi r^3$, and surface area is calculated by $a = 4 \pi r^2$)

5. Create a class named *DecimalConverter* that converts geographic coordinate values from degrees/minutes/seconds to decimal degrees. The class has three "WriteOnly" properties: *Degrees* as integer, *Minutes* as integer, and *Seconds* as Double. The function has a methods named *DecimalDegrees* that returns a Double result.

6. Create a Line class and use property structure to define two properties P1 and P2 both are Point type (i.e. the same Point class as you created in the PointCircle

project). The class must also have a "ReadOnly" property Length that returns the line length.

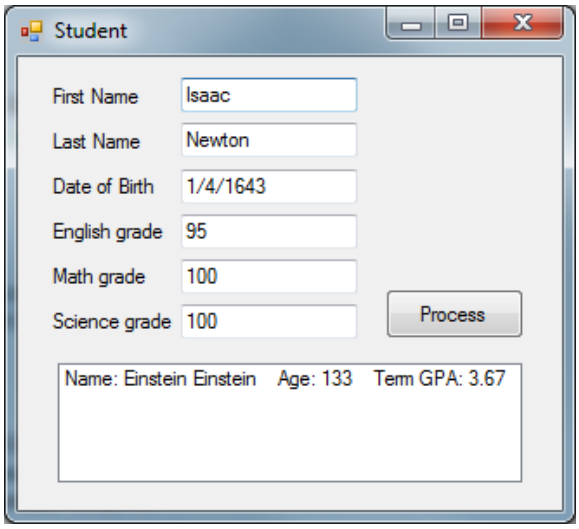
7. Modify the MPG class created in question 3 to use two property structures to implement the two properties.

8. Modify the MPG class created in question 3 to overload the constructor so that the miles and gallons members are initiated while instantiating a new MPG object.

9. Create a class named *Calculator* that inherits the AddingMachine class created in question 2. The Calculator class has two methods named *Product* and *Quotient* that calculate the product and quotient of the data members Number 1 and Number2.

10. Create a class named *Planet* that inherits the Sphere class created in question 4. The Planet class must have a property called Mass of Double type, and it must also have a method named *Density* that calculates the mass density of planets.

11. Create a VB 2010 project named Students to process student data. The project must meet the following requirements:
    1) It must have two classes: Person and Student where class Student inherits Person.
    2) The Person class must have three properties: FirstName (String), LastName (String), and DateOfBirth (Date). It must also have a method named Age that calculates the age of the person based on today's date and the person's date of birth.
    3) The Student class which inherits the Person class must have three additional Properties: English, Math, and Science, all are Integers. Each of these properties will be used to store the grade (0-100) of a corresponding course. It must also have a method named TermGPA that calculates the average GPA of the three courses.
    4) You will create a user interface similar to the screenshot below to allow users to enter data of students.
    5) In the Process button's Click event procedure, you must create a new student object from the Student class and assign proper values entered by the user to its properties. Then use appropriate methods to calculate the student's age and term GPS. Results should be displayed in a list box.



Tips:

1) To calculate age:

```
Dim today As Date = Now()
Dim years As Integer = today.Year – DateOfBirth.Year
If DateOfBirth.DayOfYear > Now.DayOfYear Then years -= 1
```

2) To calculate term GPA
   Course GPA
   A: >=90, GPA = 4
   B: >=80, GPA = 3
   C: >=70, GPA = 2
   D: >=60, GPA = 1
   F: < 60, GPA = 0
   Term GPA is the average GPA of the three courses
   You may create a private function (name it CourseGPA) in the Student class to convert the 100-point based grade to GPA, and call this function to get GPA for each course in the TermGPA method (public function).
3) The UML class diagram of the project is shown below for your reference.